Character-Level Language Modeling Using a Multi-Layer LSTM

R. Kumar
Department of Mechanical Engineering
University of Tennessee at Chattanooga
Chattanooga, TN, USA

***Abstract*—This paper presents a character-level text generator using a three-layer Long Short-Term Memory (LSTM) network in PyTorch, trained on *The Adventures of Sherlock Holmes*. The practical goal is straightforward: learn next-character patterns from a long corpus and generate 20-character outputs that look like English text. Using a 90/10 sequential train/test split, training cross-entropy decreased from 2.397 at epoch 1 to 0.719 at epoch 20, while test loss reached its best value of 1.469 at epoch 8 and then rose to 1.776 by epoch 20, consistent with late-stage overfitting. Even with that overfitting trend, sample quality improved visibly over training, moving from noisy fragments to more readable phrases such as `"a better man than t"`.**

I. Introduction

A central challenge in natural language processing is teaching models to predict what comes next in a sequence of symbols. Given the preceding characters in a sentence, a well-trained model should assign high probability to plausible continuations and low probability to implausible ones. This is the character-level language modeling problem addressed in this study.

Feed-forward networks cannot solve this problem directly: they map a fixed-size input to an output in a single pass, so every prediction is independent of every other. Text is different — the correct next character depends on everything that came before it, sometimes over many steps. Recurrent neural networks (RNNs) address this by maintaining a hidden state that is updated at each time step and passed forward, giving the model memory of prior context (Fig. 1). However, plain RNNs are notoriously difficult to train over long sequences because gradients either vanish or explode during backpropagation through time (BPTT) [1]. Long Short-Term Memory (LSTM) networks [1] resolve this through gated memory cells whose additive update pathway protects gradients from the repeated matrix multiplications that cause instability.

In this work, we train a three-layer LSTM character model on the full text of *The Adventures of Sherlock Holmes* and evaluate how well it can produce novel 20-character continuations. Section II covers the theoretical background of RNNs, BPTT, and the LSTM gating mechanism. Section III describes the corpus and preprocessing pipeline. Section IV details the model architecture and training procedure. Section V presents the generation method and the evolution of output quality. Section VI discusses overfitting, design trade-offs, and potential improvements.

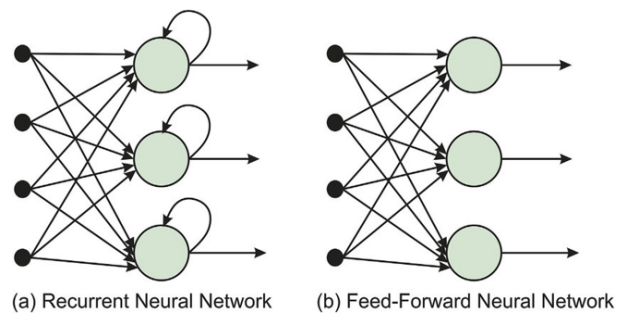

Fig. 1. Recurrent Neural Network (left) vs. feed-forward network (right). Self-loops in the RNN allow hidden state to persist across time steps, giving the model memory of prior inputs. Feed-forward networks discard all prior state at each step.

II. Theoretical Background

A. Recurrent Neural Networks

At every time step $t \in \{1, \ldots, T\}$, a vanilla RNN combines the current input $\mathbf{x}_t \in \mathbb{R}^d$ with the previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^h$ to produce a new hidden state and a prediction:

$$\mathbf{h}_t = \tanh(W_{hx}\,\mathbf{x}_t + W_{hh}\,\mathbf{h}_{t-1} + \mathbf{b}_h)\,, \tag{1}$$

$$\hat{\mathbf{y}}_t = \mathrm{softmax}(W_{yh}\,\mathbf{h}_t + \mathbf{b}_y)\,. \tag{2}$$

The hidden state $\mathbf{h}_t$ acts as a running summary of the sequence seen so far. In principle, it allows the network to condition predictions on arbitrarily long histories. In practice, the single $\tanh$ transformation makes selective retention difficult, and the gradient signal degrades rapidly over long sequences.

B. Backpropagation Through Time and Gradient Instability

Training an RNN requires computing $\partial\mathcal{L}/\partial W_{hh}$, where $\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t$ is the total sequence loss and $\mathcal{L}_t$ is the per-step cross-entropy at position t. Unrolling the recurrence and applying the chain rule yields

$$\frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \prod_{k=t+1}^{T} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}, \tag{3}$$

where each local Jacobian equals $W_{hh}^{\top}\,\mathrm{diag}(1 - \mathbf{h}_k^2)$. The product of $T-t$ such matrices causes the gradient to shrink exponentially toward zero (*vanishing gradient*) when the largest singular value of W_{hh} is below one, or to grow without bound (*exploding gradient*) when it exceeds one [2]. Either pathology

breaks long-range credit assignment, making it practically impossible for a plain RNN to learn dependencies spanning more than a few dozen time steps.

C. Long Short-Term Memory

The LSTM [1] replaces the single $\tanh$ unit with a *memory cell* $\mathbf{c}_t \in \mathbb{R}^h$. Unlike the hidden state, the cell is updated by addition rather than by a nonlinear transformation, so its gradient pathway does not involve repeated matrix multiplication — this is the mechanism that mitigates vanishing gradients. Three multiplicative gates control what information enters, persists in, and leaves the cell. Figure 2 shows the contrast between a simple RNN module and an LSTM module unrolled over time.

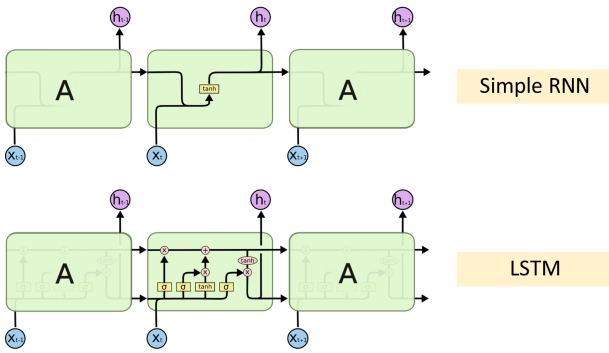

Fig. 2. Simple RNN (top) vs. LSTM (bottom) unrolled over three time steps. The simple RNN passes a single hidden state through one $\tanh$ transformation. The LSTM maintains a separate cell state (horizontal line) gated by forget (σ), input (σ, $\tanh$), and output (σ) operations, enabling stable long-range credit assignment.

Given the concatenated vector $[\mathbf{h}_{t-1}; \mathbf{x}_t]$, the four gate computations are:

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f) \quad \text{(forget gate)}, \tag{4}$$

$$\mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i) \quad \text{(input gate)}, \tag{5}$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_c) \quad \text{(cell candidate)}, \tag{6}$$

$$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o) \quad \text{(output gate)}. \tag{7}$$

The cell state and exposed hidden state are then:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{8}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \tag{9}$$

The additive update in (8) means gradients can flow backward through the $+$ operator directly without passing through W_{hh}, resolving the instability in (3). Table I summarises each gate's function.

TABLE I
LSTM GATE FUNCTIONS AND THEIR ROLES.

Gate	Activation	Role
Forget $\mathbf{f}_t$	σ	Erase stale memory from $\mathbf{c}_{t-1}$
Input $\mathbf{i}_t$	σ	Scale new candidate values to write
Candidate $\tilde{\mathbf{c}}_t$	$\tanh$	Propose new memory content
Output $\mathbf{o}_t$	σ	Filter cell state into hidden output

D. Stacking LSTM Layers

A single LSTM layer computes one level of temporal abstraction. Stacking N layers passes the hidden-state sequence of layer l as the input sequence to layer $l+1$, allowing deeper layers to model longer-range or more abstract patterns. Dropout with probability p is applied to the connections between layers during training to reduce overfitting; it is disabled at evaluation time. This study uses $N = 3$ stacked layers with $p = 0.2$.

III. Dataset and Preprocessing

A. Corpus

The training corpus is *The Adventures of Sherlock Holmes* by Arthur Conan Doyle, obtained from Project Gutenberg and cached locally as `sherlock.txt` (607 KB). The text is long enough to exhibit rich character-level statistics — word spellings, punctuation patterns, and dialogue structure — making it a suitable training corpus for a character-level language model.

B. Vocabulary and Cleaning

The raw text is filtered to a 68-symbol vocabulary: the 52 ASCII letters, 10 digits, and 6 punctuation symbols (`.,! -'`). All other characters are dropped; newlines are replaced with spaces and consecutive spaces are collapsed to one. After cleaning, the corpus contains **570,114** characters. After the one-step shift used to build input-target pairs (`training_string = training_string[:-1]`), the effective training stream length is **570,113** characters. Two lookup tables map between symbols and integer indices.

C. Sequence Construction

A character-level language model is trained to predict each character from the one before it. Formally, given the cleaned stream $c_1, c_2, \ldots, c_T$, the input and target at each position are:

$$x_t = c_t, \quad y_t = c_{t+1}, \quad t = 1, \ldots, T-1. \tag{10}$$

The stream is partitioned into non-overlapping windows of length $L = 100$, yielding **5,702** sequences in total. Each window is one-hot encoded to shape $(L, |\mathcal{V}|) = (100, 68)$, producing input tensor $X \in \mathbb{R}^{N \times 100 \times 68}$. Because 570,113 is not divisible by 100, the final sequence is a partially filled window that is zero-padded by the current implementation.

D. Train/Test Split

To assess generalization, the first 90 % of sequences are used for training and the final 10 % are held out as a test set. The split is sequential rather than random to respect the temporal ordering of the narrative. Table II shows the resulting partition.

The "sequence slots" column reports $N \times L$ positions in one-hot tensors. This is slightly larger than the cleaned corpus length because of the single padded final window.

TABLE II
SEQUENTIAL 90/10 TRAIN/TEST PARTITION OF THE CORPUS.

Split	Sequences	Sequence slots
Training (90 %)	5,131	513,100
Test (10 %)	571	57,100
Total	5,702	570,200

IV. MODEL ARCHITECTURE AND TRAINING

A. Architecture

The model, `TextGeneratorLSTM`, comprises three stacked LSTM layers with hidden dimension $h = 700$, followed by a single linear layer that maps every time step's hidden vector to a logit over the 68-symbol vocabulary:

$$\mathbf{H} = \mathrm{LSTM}_{1\to 3}(\mathbf{X}) \in \mathbb{R}^{N\times L\times 700}, \tag{11}$$

$$\hat{\mathbf{Z}} = \mathbf{H}\,W^{\top} + \mathbf{b} \in \mathbb{R}^{N\times L\times 68}, \tag{12}$$

where $\mathrm{LSTM}_{1\to 3}$ denotes the three-layer stack applied sequentially. Because `nn.Linear` broadcasts over the time axis, the same projection is applied at each of the $L = 100$ positions without additional reshaping. The total number of trainable parameters is **10,054,868**.

B. Objective Function

At each time step and for each sequence in a mini-batch, the model produces a 68-dimensional logit vector. These are flattened to shape $(NL, 68)$ and compared against the $(NL,)$ target class indices using categorical cross-entropy:

$$\mathcal{L} = -\frac{1}{NL}\sum_{n=1}^{N}\sum_{t=1}^{L}\log p\Big(y_t^{(n)} \mid x_{\le t}^{(n)}\Big)\,. \tag{13}$$

A randomly initialized model assigns uniform probability to all 68 symbols, giving a baseline loss of $\ln(68) \approx 4.22$. Any loss below this indicates that the model has learned non-trivial structure from the corpus.

C. Optimizer and Training Protocol

The Adam optimizer [3] is used with default hyperparameters ($\alpha = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$) and a mini-batch size of 12. Training sequences are shuffled at the start of each epoch. The model is trained for 20 epochs; checkpoints are saved at epochs 10 and 20 for later analysis.

D. Training and Test Loss

Figure 3 shows the per-epoch training and test loss. Training loss fell monotonically from **2.397** at epoch 1 to **0.719** at epoch 20, a 70 % reduction from the starting value and well below the random baseline of 4.22.

Test loss behaves differently in two distinct phases. In the first phase (epochs 1–8), test loss decreases from 2.054 to a minimum of **1.469**, showing genuine generalization improvement. In the second phase (epochs 9–20), test loss rises steadily to **1.776** while training loss continues to fall, producing a train-test gap that widens from $+0.43$ nats at epoch 9 to $+1.06$ nats at epoch 20. This divergence is the signature of overfitting: the model is memorizing training sequences at the expense of held-out performance.

One detail is worth noting: at epoch 1, the test loss (2.054) is lower than the training loss (2.397). This is expected and is caused by the inter-layer dropout applied during training — dropout randomly zeroes hidden units, increasing the training-time loss relative to the evaluation-time loss where dropout is disabled. From epoch 2 onward, as the model begins to fit the training data, the usual ordering (train loss $<$ test loss) is restored and the gap grows.

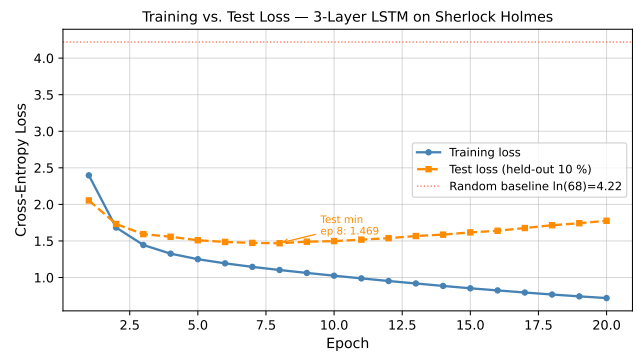

Fig. 3. Per-epoch training loss (solid blue, circles) and test loss (dashed orange, squares) over 20 epochs. The dotted red line marks the random-initialization baseline $\ln(68) \approx 4.22$. Test loss is minimized at epoch 8 (1.469); the widening gap thereafter indicates overfitting.

V. TEXT GENERATION

A. Generation Method

After each training epoch, the model generates a 20-character sequence using autoregressive greedy decoding. A seed character is selected with a fixed random seed to make samples comparable across epochs. At each step k, the model receives the growing input buffer and selects the highest-probability next character:

$$i_{k+1} = \arg\max_{j\in\mathcal{V}}\ \hat{z}_{k,\,j}\,. \tag{14}$$

The selected character is appended to the buffer and fed back as the next input; this repeats until exactly 20 characters have been produced.

B. Evolution of Output Quality

Table III shows the generated sample at six representative epochs, alongside the corresponding train and test losses.

At epoch 1, the output mixes a noisy fragment (`shand`) with common words (`the`, `he`, `has`), reflecting a crude approximation of the character distribution. By epoch 4, full English words appear in grammatically plausible combinations (`that the man who wa[s]`). At epoch 8, the point of best held-out performance, the model produces a semantically coherent incomplete phrase (`the strange possess[ion]`). Epochs 12–20 continue to show improvement in output fluency, despite rising test loss, because

TABLE III
GENERATED 20-CHARACTER SAMPLES AT REPRESENTATIVE EPOCHS. ALL OUTPUTS ARE REAL MODEL PREDICTIONS RECORDED DURING TRAINING.

Epoch	Train	Test	Generated Sample
1	2.397	2.054	`er the shand he has`
4	1.326	1.558	`that the man who wa`
8	1.103	1.469	`the strange possess`
12	0.952	1.538	`the coronet and a b`
16	0.824	1.639	`a large number of h`
20	0.719	1.776	`a better man than t`

greedy decoding on a more confident model tends to produce longer runs of common English words regardless of generalization. The epoch 20 sample (`a better man than t[he/hat]`) is a grammatically complete phrase fragment consistent with Conan Doyle's prose style.

VI. DISCUSSION

At first glance, the results may look contradictory: test loss rises after epoch 8, but generated text often sounds more fluent at epoch 20. In practice, both observations can be true at the same time. Test loss measures the model's average surprise on unseen sequences, which is sensitive to low-frequency patterns and rare words. Greedy decoding, by contrast, always selects the most probable character and therefore gravitates toward the most common words and transitions in the corpus — the very patterns the model memorizes first. A model that overfits common phrases may still appear fluent under greedy decoding while scoring worse on held-out perplexity. Where readable output is the goal, the epoch-20 model is the better deliverable; where generalization matters, the epoch-8 checkpoint is preferable.

This overfitting pattern is not surprising given the model's capacity relative to the training data. With 10 million parameters trained on 5,131 sequences of 100 characters, the model sees roughly 513,000 character positions per epoch. Inter-layer dropout is the only regularizer, and it is not strong enough to prevent memorization once the model has seen the same windows several times. Increasing the stride overlap — for example, using a stride of 10 rather than 100 — would produce approximately 58,000 training sequences from the same corpus, providing far more context variety per epoch and likely delaying the onset of overfitting considerably.

One further limitation of the generation procedure is that greedy $\arg\max$ selection is fully deterministic: any two runs from the same seed character produce identical output regardless of how many times the model is queried. Temperature-scaled sampling, $p_j \propto \exp(\hat{z}_j/\tau)$, provides a simple alternative without retraining. Setting $\tau < 1$ sharpens the distribution toward the model's most confident predictions, while $\tau > 1$ flattens it to encourage more varied and sometimes surprising output.

VII. CONCLUSION

This paper presented a three-layer, 10-million-parameter LSTM character model trained on *The Adventures of Sherlock Holmes*. Training cross-entropy loss decreased from 2.397 to 0.719 over 20 epochs, far below the random-initialization baseline of 4.22. A 90/10 train/test split revealed clear overfitting from epoch 9 onward, with test loss rising from its minimum of 1.469 (epoch 8) to 1.776 (epoch 20). Despite this, generated 20-character samples improved in perceptual quality throughout training, producing coherent English phrase fragments by epoch 8 and beyond. The analysis suggests that the epoch 8 checkpoint is optimal for generalization, while the epoch 20 checkpoint produces more fluent output under greedy decoding.

APPENDIX A

The Google Colab notebook for this assignment is attached below:

- LSTM-NLP

The GitHub repository for the project can be found at:

- github

The YouTube video for this assignment can be found at:

- YouTube

REFERENCES

1 S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

2 I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

3 A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.